ジオメトリック・インテリジェンス

ケーススタディ補助文書 03

実装スクリプト集

― 国王陛下の多様体：英国国家レジリエンス多様体の構築 ―

本書は、ケーススタディ・ノベル「国王陛下の多様体」に登場するコードを整理し、
背景、解説、実装上の注意、参照実装を示す。

Étale Cohomology

2026 年 10 月

License: CC BY 4.0

目次

1　はじめに　2

2　命題 1.1 の確認（第 2 章）　2

3　リー括弧（第 3・5 章）　2

4　パーシステント・ホモロジー（第 9 章）　3

5　層ラプラシアン（第 11 章）　3

6　手術計画（第 12 章）　4

7　ゲーム理論の利得（第 12 章）　4

8　シンプレクティック積分（第 13 章）　4

9　形式検証と ZKML（第 15 章）　5

付録 A　参照実装　6

第1章　はじめに

本書は、本編に登場するコードを場面の順に整理し、それぞれの背景、解説、実装上の注意を示す。本編のコードは、物語の中で計算の流れを示すための抜粋であり、ライブラリの関数名（`lie_bracket`、`construct_whitehall_sheaf` など）は説明のための架空の名前である。実際に動かせる最小限の例は、各節の参照実装と付録 A に示した。いずれも実行し、出力を確認している。

第2章　命題 1.1 の確認（第 2 章）

```
print("条件1：MSE = 0.0048 → 合格 ✓")
print("条件2：KNN保存率 = 0.927 → 合格 ✓")
print("条件3：最小特異値 = 0.0032 → 合格 ✓")
print("★ 英国国家レジリエンス多様体 ―― 23次元 ―― 構築完了 ★")
```

解説：命題 1.1 の 3 つの条件の確認結果である。第 1 回試行では、Brexit の前後でデータが 2 つの島に分かれ、KNN 保存率が 0.68 にとどまった。EU 離脱度の追加と Neural ODE による補間で、第 3 回試行で全条件を満たした。

実装上の注意：KNN 保存率は、元の空間と潜在空間のそれぞれで k 近傍を求め、重なりの割合を平均して計算する。構造変化の前後で別々に計算し、どちらか一方だけが低くないかも確かめる。補間した時点（印を付けた時点）は、評価から除いた値も併記するとよい。

第3章　リー括弧（第 3・5 章）

```
bracket_russia_china = lie_bracket(X_russia_threat, X_china_threat, z_current)
print(f"リー括弧 [ロシア, 中国] = {np.linalg.norm(bracket_russia_china):.2f}")
```

解説：ロシアの脅威の場と中国の脅威の場のリー括弧を、現在の英国の状態 z で計算し、その大きさ（ノルム）を示している。本編の値は 5.87 である。第 5 章の [エネルギー危機, 中国金融浸透] = 6.14 も同じ計算である。

実装上の注意：リー括弧は $[X, Y](z) = J_Y(z)X(z) - J_X(z)Y(z)$ で計算する。ヤコビ行列は自動微分で求めるのが確実だが、確認のためには中心差分でもよい。ベクトル場は、VAE の潜在空間の上で学習した場（各脅威の時系列から推定した速度場）として与える。次の参照実装は、理論値がわかる簡単な場で、計算が正しいことを確かめる例である。

参照実装：中心差分によるリー括弧

```
import numpy as np
# ベクトル場 X, Y のリー括弧 [X, Y](z) = J_Y(z) X(z) - J_X(z) Y(z) を、中心差分のヤコビ行列で計算する
def jacobian(F, z, eps=1e-6):
    n = len(z); J = np.zeros((n, n))
    for k in range(n):
        e = np.zeros(n); e[k] = eps
        J[:, k] = (F(z + e) - F(z - e)) / (2*eps)
    return J
```

```
def lie_bracket(X, Y, z):
    return jacobian(Y, z) @ X(z) - jacobian(X, z) @ Y(z)
# 例：平面の回転の場 X と、x 方向の一様な場 Y
X = lambda z: np.array([-z[1], z[0]])
Y = lambda z: np.array([1.0, 0.0])
z = np.array([0.3, -0.2])
print("[X, Y](z) =", np.round(lie_bracket(X, Y, z), 6))   # 理論値は (0, -1)
# 可換な場どうしでは 0 になることの確認
Z = lambda z: np.array([2.0, 0.5])
print("[Y, Z](z) =", np.round(lie_bracket(Y, Z, z), 6))
```

出力は `[X, Y](z) = [ 0. -1.]`、`[Y, Z](z) = [0. 0.]` で、それぞれ理論値と一致する。一様な場どうしのリー括弧は 0 になる（平行四辺形が閉じる）。

第4章　パーシステント・ホモロジー（第 9 章）

```
crisis_points = np.array([z_energy, z_finance, z_ni, z_cable] + related_events)
# 関連する19の小さな事象を加える
result = ripser(crisis_points, maxdim=1)
```

解説：4 つの主な危機と、関連する 19 の小さな事象の潜在空間の座標から、1 次元の穴（輪）を探す。結果は持続性 0.89 の輪 1 つである。

実装上の注意：`ripser` は Python のライブラリで、`result['dgms'][1]` に 1 次元の持続性図が入る。持続性（消える半径 − 現れる半径）が大きい輪だけを構造と見なす。頑健性は、点の座標に小さなノイズを加えて何度も計算し、輪が繰り返し現れることで確かめる。距離には、潜在空間のユークリッド距離ではなく、可能であれば多様体の上の測地線距離を使う。

第5章　層ラプラシアン（第 11 章）

```
sheaf = construct_whitehall_sheaf(personnel, access_logs, data_flows)
L = sheaf.laplacian()
x = sheaf.get_global_data_vector()  # 組織全体のデータを 1 つのベクトルに並べる
Lx = L @ x                          # 層ラプラシアンを一度だけ作用させる
for person in cleared_personnel:
    inconsistency = np.linalg.norm(sheaf.block(Lx, person))  # その職員に対応するブロックの大きさ
    if inconsistency > threshold:
        print(f"不整合：{person.name}（{person.dept}）スコア={inconsistency:.2f}")
```

解説：組織全体のデータを 1 つのベクトルに並べ、層ラプラシアンを一度だけ作用させて、各職員に対応するブロックの大きさをスコアとしている。本編のスコアは 6.89 である。職員一人分のデータに組織全体の行列を掛けると、行列とベクトルの大きさが合わないので注意する（最小限の参照実装は付録 A.2）。

実装上の注意：閾値は、正常な期間のスコアの分布から決める。スコアは「調べるべき場所」を示すだけで、不正の認定ではない。氏名は出力に含めず、法にもとづく調査・捜査の手続きの中でだけ照合する。

第6章　手術計画（第 12 章）

```
surgery_plan_uk = {
    'cut': [
        'ロシア関連エネルギー取引の残存分の完全遮断',
        '戦略的セクターへの投資審査の強化（2021年国家安全保障・投資法）',
        'ウィンザー・フレームワークの円滑な運用＋英EU間のSPS（衛生植物検疫）協定',
    ],
    'glue': [
        'エネルギー：原子力新設加速（Sizewell C）＋北海洋上風力＋水素',
        '金融：CPTPP諸国＋中東との金融連携強化（シティの再定位）',
        '防衛：AUKUS加速＋英国主導の統合遠征部隊（JEF）＋英EU安全保障・防衛パートナーシップ',
        'サプライチェーン：半導体（英国発の設計IP）＋医薬品国内製造',
    ],
}
```

解説：手術計画を、切る項目（cut）と貼る項目（glue）の一組として表した設計図である。cut の 3 行目は、北アイルランドをめぐる摩擦が他の危機と連動して輪を太らせる「経路」を断つ手立てである。

実装上の注意：計算の上では、危機の点どうしの依存関係を重み付きグラフ（または単体複体）として持ち、cut は辺の重みを下げる（または辺を除く）操作、glue は新しい辺や点を加える操作として表す。手術の後は、(1) パーシステント・ホモロジーを再計算して輪が消えたか、(2) 曲率を再計算して谷が浅くなったか、(3) 命題 1.1 の条件が保たれているか、の 3 つを確かめる。数学の手術（$S^{k}\times D^{n-k}$ を $D^{k+1}\times S^{n-k-1}$ に置き換える操作）は、この依存関係のネットワークの組み替えとして近似的に実装される。

第7章　ゲーム理論の利得（第 12 章）

本編にコードは載っていないが、計算の流れは次のとおりである。各戦略について、(1) 英国の状態が移る先の点を多様体の上で推定し、(2) その点のスカラー曲率（深さ）と、(3) 現在地からその点までの測地線の長さを求め、(4) 両者を組み合わせて利得とする。

実装上の注意：測地線の長さは、潜在空間の直線距離ではなく、引き戻し計量 $g = J^{\top}J$（J はデコーダのヤコビ行列）のもとで、経路に沿って $\sqrt{\dot z^{\top} g\, \dot z}$ を積分して求める。曲がった地形を斜めから描いた図では、見かけの長さと測地線の長さが一致しないことに注意する。

第8章　シンプレクティック積分（第 13 章）

解説：10 年間のシミュレーションでは、シンプレクティック積分で、系が保つべき量のずれを約 1 億分の 1 にとどめた。

実装上の注意：普通の方法（陽的オイラー法）では、ずれが時間とともに積み重なる。次の参照実装は、調和振動子で両者を比べる最小限の例である。

参照実装：陽的オイラー法とリープフロッグ法の比較

```
import numpy as np
# 調和振動子 H = (p^2 + q^2)/2 を 1万ステップ進めて、エネルギーのずれを比べる
def euler(q, p, h):          # 普通の（陽的）オイラー法
    return q + h*p, p - h*q
def leapfrog(q, p, h):       # シンプレクティックなリープフロッグ法（Störmer–Verlet）
    p_half = p - 0.5*h*q
    q_new = q + h*p_half
    p_new = p_half - 0.5*h*q_new
    return q_new, p_new
H = lambda q, p: 0.5*(p*p + q*q)
for name, step in [("euler", euler), ("leapfrog", leapfrog)]:
    q, p, h = 1.0, 0.0, 0.01
    H0 = H(q, p); worst = 0.0
    for _ in range(10000):
        q, p = step(q, p, h)
        worst = max(worst, abs(H(q, p) - H0)/H0)
    print(f"{name:9s} 最大の相対誤差 |ΔH/H| = {worst:.2e}")
```

出力は、陽的オイラー法で最大の相対誤差 1.72（エネルギーが 2 倍以上にふくらむ）、リープフロッグ法で 2.50×10^{-5} である。リープフロッグ法のずれは刻み幅 h の 2 乗に比例し、刻み幅を小さくするか、より高次のシンプレクティック法を使えば、さらに小さくできる。いずれの場合も、保証されるのは計算が崩れないことであり、未来の予測が当たることではない。

第9章　形式検証と ZKML（第 15 章）

解説：本編では、Coq でデコーダのリプシッツ定数の上限（$0.74 \le 0.8$）を証明し、Z3 で約 2,000 の規則が同時に満たされることを確かめ、Agda の型で人間の承認なしには命令を組み立てられないことを保証した。ZKML の証明は約 580 万ゲートで、検証は 1.9 秒である。

実装上の注意：ニューラルネットワークのリプシッツ定数の上限は、各層の重み行列の作用素ノルム（最大特異値）の積で抑えられる（活性化関数のリプシッツ定数が 1 以下の場合）。この上限は粗いことが多いので、より精密な手法と併用する。ZKML では、モデルの重みと計算を有限体の上の算術回路に変換するため、実数の計算を固定小数点で近似する。近似による誤差の範囲も、証明の対象に含めて設計する。

付録 A　参照実装

本付録のコードは、本文の「実装上の注意」で述べた点を示すための最小限の例であり、本編の数値を再現するものではない。いずれも実際に実行し、出力を確認した。

A.1　集計・小さなセルの秘匿・差分プライバシー

```
import numpy as np

def aggregate_monthly(records, months, categories, k_min=5, epsilon=1.0, seed=0):
    """個別の記録（月, 区分）を月次・区分ごとの件数にまとめ、小さなセルを秘匿し、
    ラプラス・ノイズで差分プライバシーを付ける（1 件の増減で件数は最大 1 変わる）。"""
    rng = np.random.default_rng(seed)
    counts = np.zeros((len(months), len(categories)))
    for m, c in records:
        counts[months.index(m), categories.index(c)] += 1
    suppressed = counts < k_min                       # 小さなセルの秘匿
    noisy = counts + rng.laplace(0.0, 1.0 / epsilon, counts.shape)
    noisy[suppressed] = np.nan                        # 伏せた区分は欠測として扱う
    return np.round(noisy, 1), suppressed

months = ['2026-04', '2026-05', '2026-06']
cats = ['接触', '訪問', '通信']
table = {('2026-04', '接触'): 12, ('2026-04', '訪問'): 9, ('2026-04', '通信'): 3,
         ('2026-05', '接触'): 15, ('2026-05', '訪問'): 11, ('2026-05', '通信'): 8,
         ('2026-06', '接触'): 21, ('2026-06', '訪問'): 2,  ('2026-06', '通信'): 10}
records = [key for key, n in table.items() for _ in range(n)]   # 個別の記録（例）
agg, sup = aggregate_monthly(records, months, cats)
print(agg)
print('秘匿したセルの数:', int(sup.sum()))
```

実行結果（ノイズは乱数の種を固定して生成）：

```
[[12.3  8.4  nan]
 [11.6 12.   9.7]
 [21.2  nan 10.1]]
秘匿したセルの数: 2
```

件数が 5 未満の区分（2026 年 4 月の「通信」3 件、2026 年 6 月の「訪問」2 件）は伏せられ、それ以外の件数にはノイズが加わっている。多様体には、このような集計指標だけが入る。

A.2　層ラプラシアンと頂点ごとのスコア

```
import numpy as np

def sheaf_laplacian(n_nodes, stalk_dim, edges):
```

```
    """edges: [(u, v, F_u, F_v)]。F_u, F_v は頂点のデータを辺の上で比べる制限写像（行列）。"""
    rows = []
    for u, v, Fu, Fv in edges:
        r = np.zeros((Fu.shape[0], n_nodes * stalk_dim))
        r[:, u*stalk_dim:(u+1)*stalk_dim] = -Fu
        r[:, v*stalk_dim:(v+1)*stalk_dim] = Fv
        rows.append(r)
    delta = np.vstack(rows)          # 余境界作用素 δ
    return delta, delta.T @ delta    # 層ラプラシアン L = δ^T δ

def node_scores(L, x, n_nodes, stalk_dim):
    """全体のデータ x に L を一度だけ作用させ、頂点ごとのブロックの大きさをスコアにする。"""
    Lx = (L @ x).reshape(n_nodes, stalk_dim)
    return Lx, np.linalg.norm(Lx, axis=1)

# 3 部署の演習問題：A(営業), B(物流), C(経理)。各部署は（数量, 単価）を持つ
I = np.eye(2)
edges = [(0, 1, I, I), (1, 2, I, I), (2, 0, I, I)]   # 両端の記録は一致すべき
x = np.array([10, 50,  10, 50,  10, 80], float)
delta, L = sheaf_laplacian(3, 2, edges)
Lx, score = node_scores(L, x, 3, 2)
print("L x =", Lx.tolist())
print("スコア =", np.round(score, 1).tolist(), " 整合性半径 =", round(float(np.linalg.norm(delta @ x)), 2))
```

実行結果：

```
L x = [[0.0, -30.0], [0.0, -30.0], [0.0, 60.0]]
スコア = [30.0, 30.0, 60.0]  整合性半径 = 42.43
```

組織全体のデータを 1 つのベクトルに並べ、層ラプラシアンを一度だけ作用させて、頂点ごとのブロックの大きさをスコアにしている。補助文書 02 第 12.7 節の演習問題と同じ結果である。